Imagine you want to check the attendee list to see if a particular person is attending. How do you write a program for that? You might think to start by using the `search` method on a string that you learned in chapter 2 (remember, `search` returns the value `-1` if it doesn't find a match). Consider a `contains` function that searches a string of comma-separated words:

```
contains = (text, word) -> (text.search word) isnt -1
randomWords = 'door,comeuppance,jacuzzi,tent,hippocampus,gallivant'

contains randomWords, 'tent'
# true
```

Remember, the isnt keyword is the CoffeeScript alias for !==.

Unfortunately, this `contains` function returns the wrong value for the word *camp* because the word *hippocampus* contains the word *camp*:

```
contains randomWords, 'camp'
#true
```

In case you missed the memo, false positives are not a design feature. You already have a function that splits these strings into individual words, so back to word counting—here's a one-liner:

```
wordCount = (text, delimiter) -> text.split(delimiter).length
```

Use this as the starting point for a correct `contains` function.

Reuse

To avoid duplication, you want to reuse this `wordCount` function, but sadly, it currently does too much for you to be able to reuse it. You need a function that splits the string into an array. So, take out the split and make it a separate function:

```
split = (text, delimiter) -> text.split delimiter
```

Now you can define `wordCount` as a function that invokes `split`:

```
wordCount = (text, delimiter) ->
  words = split(text, delimiter)
  words.length
```